

CounterX – Inventory Management Module

Feature Implementation Summary

Project

CounterX – AI Powered Virtual Cashier Platform

Module

Inventory Management

Technology Stack

- Java 21
 - Spring Boot 3.x
 - Spring Data JPA
 - Hibernate
 - MySQL 8
 - Maven
-

Overview

The Inventory Management Module is designed to manage restaurant inventory, track stock movements, automate inventory deductions based on completed orders, maintain inventory audit history, and provide dashboard analytics for inventory monitoring.

The module follows a Modular Monolith architecture and is built using enterprise-grade Spring Boot practices.

Features Implemented

1. Inventory Management

Implemented APIs

- Create Inventory Item
- Update Inventory Item
- Get Inventory By ID
- Get All Inventory
- Soft Delete Inventory

Capabilities

- Inventory item creation

- Inventory updates
 - Inventory search support
 - Batch number tracking
 - Expiry date tracking
 - Category management
 - Unit management
 - Soft delete support
-

2. Stock Management

Stock Addition

Allows administrators to add stock for existing inventory items.

Example:

Paneer Stock: 50000g → 55000g

Stock Reduction

Allows manual stock reduction.

Example:

Paneer Stock: 55000g → 54000g

Automatic Stock Status Calculation

Supported statuses:

- IN_STOCK
- LOW_STOCK
- OUT_OF_STOCK
- EXPIRED

Status updates automatically whenever stock changes.

3. Inventory History Tracking

Automatic Audit Trail

Every inventory movement creates a history record automatically.

Tracked Events:

- STOCK_ADDED
- STOCK_REDUCED

- ORDER_DEDUCTION
- MANUAL_ADJUSTMENT
- ITEM_EXPIRED
- INVENTORY_CREATED
- INVENTORY_UPDATED
- INVENTORY_DELETED

History Information Stored

- Previous Quantity
- Changed Quantity
- New Quantity
- Action Type
- Remarks
- Created By
- Created Date

This provides complete inventory auditability.

4. Menu Ingredient Mapping

Implemented menu-to-ingredient mapping.

Example

Paneer Butter Masala

Ingredients:

- Paneer = 250g
- Tomato = 150g
- Onion = 100g
- Butter = 50g

The system stores ingredient consumption rules for every menu item.

5. Automatic Order-Based Inventory Deduction

Business Flow

Order Completed → Payment Successful → Fetch Recipe → Calculate Ingredient Consumption →
Validate Stock → Deduct Inventory → Create Inventory History → Commit Transaction

Example

Paneer Butter Masala × 2

Deduction:

- Paneer = 500g
- Tomato = 300g
- Onion = 200g
- Butter = 100g

Inventory updates automatically after successful order completion.

6. Transaction Management

All inventory deduction operations are transactional.

Benefits

If any ingredient has insufficient stock:

- Entire transaction rolls back
- No inventory updates occur
- No history records are created

Ensures inventory consistency.

7. FIFO Batch Consumption Strategy

Implemented First-In-First-Out (FIFO) inventory consumption.

Consumption Order:

Oldest Expiry Date → Consumed First

Benefits:

- Reduces wastage
 - Improves inventory rotation
 - Supports restaurant best practices
-

8. Low Stock Monitoring

System automatically identifies:

Low Stock Items

Condition:

Available Stock $\leq$ Minimum Stock

Used for:

- Inventory alerts
 - Dashboard reporting
 - Purchase planning
-

9. Expiry Management

Implemented expiry tracking support.

Features:

- Expiry date monitoring
 - Expired stock identification
 - Automatic status update to EXPIRED
 - Inventory history creation
-

10. Dashboard Analytics

Dashboard APIs Implemented

Inventory Value

Current inventory worth calculation.

Formula:

Available Stock × Price Per Unit

Low Stock Count

Total inventory items below threshold.

Out Of Stock Count

Total inventory items with zero stock.

Expiring Products Count

Items nearing expiry.

Daily Consumption

Ingredient consumption for current day.

Weekly Consumption

Ingredient consumption for last 7 days.

Monthly Consumption

Ingredient consumption for last 30 days.

Top Consumed Items

Most frequently consumed ingredients.

11. Exception Handling

Custom exceptions implemented.

ResourceNotFoundException

Used when:

- Inventory not found
- Recipe not found
- Invalid inventory references

InsufficientStockException

Used when:

- Stock requested exceeds available quantity

Global Exception Handling

Standardized API error responses.

12. API Response Standardization

All APIs return a standard response structure.

Example:

```
{ "success": true, "message": "Inventory fetched successfully", "data": {}, "timestamp":  
"2026-06-02T10:00:00" }
```

Benefits:

- Consistent API responses
- Easier frontend integration
- Better debugging

13. Database Features

Tables Implemented

1. inventory
2. inventory_history
3. menu_ingredient_mapping

Additional Features

- Primary Keys
- Foreign Keys
- Indexes
- Soft Delete
- Audit Fields
- Optimistic Locking (@Version)

14. Repository Layer Features

Implemented custom queries:

- Find Low Stock Items
- Find Expiring Items
- Find By Category
- Find By Item Name
- Search Inventory
- Find By Batch Number
- Inventory Consumption Reports
- Dashboard Analytics Queries

Business Benefits

Operational Benefits

- Real-time inventory visibility
- Automated stock deduction
- Reduced manual effort
- Improved inventory accuracy

Financial Benefits

- Reduced stock wastage
- Better inventory planning
- Lower inventory holding costs

Audit Benefits

- Complete inventory traceability
 - Historical stock movement tracking
 - Compliance-ready audit records
-

Current Module Status

Completed

- ✓ Inventory CRUD
- ✓ Stock Management
- ✓ Inventory History Automation
- ✓ Menu Ingredient Mapping
- ✓ Order-Based Inventory Deduction
- ✓ FIFO Consumption Strategy
- ✓ Dashboard Analytics
- ✓ Exception Handling
- ✓ REST APIs
- ✓ Database Design
- ✓ Repository Layer
- ✓ Service Layer

Ready for Integration

- Order Module
 - Billing Module
 - Dashboard Module
 - AI Analytics Module
-

Conclusion

The Inventory Management Module provides a complete restaurant inventory solution capable of handling stock management, inventory auditing, automated order-based deductions, dashboard

reporting, and inventory analytics. The implementation follows enterprise-grade Spring Boot standards and is designed for scalability, maintainability, and future integration with other CounterX modules.